time you don't have to worry about this because the browser will take care of it for you, and Rails will decode it back when it receives it, but if you ever find yourself having to send those requests to the server manually you have to keep this in mind.

The value of `params[:ids]` will now be `["1", "2", "3"]`. Note that parameter values are always strings; Rails makes no attempt to guess or cast the type.
Values such as `[nil]` or `[nil, nil, ...]` in `params` are replaced with `[]` for security reasons by default. See Security Guide for more information.

To send a hash you include the key name inside the brackets:

```
<form accept-charset="UTF-8" action="/clients" method="post">
  <input type="text" name="client[name]" value="Acme" />
  <input type="text" name="client[phone]" value="12345" />
  <input type="text" name="client[address][postcode]" value="12345" />
  <input type="text" name="client[address][city]" value="Carrot City" />
</form>
```

When this form is submitted, the value of `params[:client]` will be `{ "name" => "Acme", "phone" => "12345", "address" => { "postcode" => "12345", "city" => "Carrot City" } }`. Note the nested hash in `params[:client][:address]`.
Note that the `params` hash is actually an instance of `ActiveSupport::HashWithIndifferentAccess`, which acts like a hash but lets you use symbols and strings interchangeably as keys.

4.2 JSON parameters

If you're writing a web service application, you might find yourself more comfortable accepting parameters in JSON format. If the "Content-Type" header of your request is set to "application/json", Rails will automatically convert your parameters into the `params` hash, which you can access as you would normally.
So for example, if you are sending this JSON content:

```
{ "company": { "name": "acme", "address": "123 Carrot Street" } }
```

You'll get `params[:company]` as `{ "name" => "acme", "address" => "123 Carrot Street" }`.
Also, if you've turned on `config.wrap_parameters` in your initializer or calling `wrap_parameters` in your controller, you can safely omit the root element in the JSON parameter. The parameters will be cloned and wrapped in the key according to your controller's name by default. So the above parameter can be written as:

```
{ "name": "acme", "address": "123 Carrot Street" }
```

And assume that you're sending the data to `CompaniesController`, it would then be wrapped in `:company` key like this:

```
{ name: "acme", address: "123 Carrot Street", company: { name: "acme",
address: "123 Carrot Street" } }
```

You can customize the name of the key or specific parameters you want to wrap by consulting theAPI documentation
Support for parsing XML parameters has been extracted into a gem named `actionpack-xml_parser`

4.3 Routing Parameters

The `params` hash will always contain the `:controller` and `:action` keys, but you should use the methods `controller_name` and `action_name` instead to access these values. Any other parameters defined by the routing, such as `:id` will also be available. As an example, consider a listing of clients